

The Hive: Comprehensive Feature Specification

An Exhaustive Breakdown of Architecture, Agents, UI, and Outputs

This document provides a maximal, feature-by-feature breakdown of The Hive. It is intended to serve as the ultimate reference manual, covering the underlying artificial intelligence architecture, the individual agent capabilities, the dual-interface visual design, and the highly specific output artifacts generated by the system during adversary emulation.

I. Core Orchestration Engine & Architecture

At the heart of The Hive is a heavily optimized, asynchronous orchestration engine. Unlike legacy tools that execute static YAML scripts top-to-bottom, The Hive uses a Directed Acyclic Graph (DAG) driven by local Large Language Models (LLMs). This means the system reasons about the network environment in real-time.

- **Local Privacy-First AI:** The platform integrates natively with Ollama (specifically targeting Llama 3.1 70B). It performs all reasoning locally on consumer GPU hardware. Zero bytes of sensitive threat intelligence or network topology are ever sent to cloud providers like OpenAI or Anthropic.
- **DAG Execution Model:** The 8 AI agents do not run randomly. They operate within a strict dependency graph. The output of the Intel Agent strictly forms the input of the Planner Agent. The planner's output dictates the exact operational flow of the 5 execution agents.
- **State Persistence:** Between every single agent execution, the complete internal state, memory, and findings are dumped to intermediate JSON files. If a simulation is interrupted, it can resume perfectly from the last known state without losing forensic data.

II. The Intelligence Phase: Intel Agent

The first step in any adversary emulation is understanding the adversary. The Intel Agent bridges the massive gap between unstructured human language (blog posts, PDFs) and machine-executable code.

- **Unstructured Text Parsing:** The Intel Agent uses Natural Language Processing (NLP) to read raw threat reports (e.g., a Mandiant report on APT29) and identify sentences describing malicious behavior.
- **MITRE ATT&CK Mapping:** It cross-references identified behaviors against the massive MITRE ATT&CK Enterprise Matrix (containing over 700 techniques) and extracts the exact IDs (e.g., T1059.001 for PowerShell).
- **Confidence Scoring:** It assigns a mathematical confidence score to each extracted technique, ensuring that low-probability or ambiguous intelligence is flagged before execution.

III. The Planning Phase: Planner Agent

A list of techniques is useless if executed in the wrong order. You cannot exfiltrate data before you gain access. The Planner Agent acts as the Red Team Lead, organizing the attack.

- **14-to-5 Phase Compression:** The standard MITRE framework has 14 tactical phases. The Planner compresses these into a highly actionable 5-phase operational Kill Chain: Recon, Access, Persistence, Movement, and Exfiltration.
- **Dependency Resolution:** The Planner evaluates the prerequisites of every technique. If a technique requires Administrator privileges, the Planner schedules privilege escalation techniques immediately prior to it.
- **Atomic Mapping:** The Planner validates which extracted techniques have corresponding live-fire execution scripts available via the Atomic Red Team database.

IV. Execution Phase: Reconnaissance Agent

The Recon Agent is the first operator to touch the target network. Its goal is situational awareness.

- **Network Sweeping:** Executes commands like ARP sweeps and ICMP pings to build a map of the local subnet.
- **Topology Generation:** In simulation mode, it uses LLM logic to hallucinate a highly realistic corporate network topography (e.g., 10.0.0.0/24 containing Domain Controllers, File Servers, and Workstations).
- **Defense Discovery:** Actively looks for running EDR processes (CrowdStrike, SentinelOne) to inform subsequent agents of potential roadblocks.

V. Execution Phase: Initial Access Agent

The Access Agent breaches the perimeter. It focuses heavily on user execution and fileless malware delivery.

- **Payload Staging:** Simulates dropping malicious LNK files, macro-enabled Word documents, or disguised PDFs onto the filesystem.
- **Memory Injection:** Executes Base64 encoded PowerShell payloads designed to run purely in RAM, avoiding disk-based Antivirus signatures.

VI. Execution Phase: Persistence Agent

The Persistence Agent ensures the Red Team does not lose their foothold if the target machine reboots.

- **Registry Manipulation:** Modifies HKCU and HKLM Run/RunOnce keys to execute malicious binaries on user login.
- **Scheduled Tasks:** Creates hidden Windows Scheduled Tasks configured to run as SYSTEM, ensuring highest-level privileges upon reboot.

VII. Execution Phase: Lateral Movement Agent

The Lateral Movement Agent focuses on spreading the infection from the initial patient-zero to the crown jewels.

- **Credential Access:** Simulates LSASS memory dumping (Mimikatz style) to steal NTLM hashes or Kerberos tickets.
- **Pass-The-Hash Execution:** Uses the stolen credentials to authenticate to remote SMB shares (C\$, ADMIN\$) and execute code via Windows Management Instrumentation (WMI) or PsExec.

VIII. Execution Phase: Exfiltration Agent

The Exfiltration Agent completes the attack by stealing the data.

- **Data Staging:** Finds sensitive files (e.g., HR databases, source code), compresses them into ZIP archives, and encrypts them with a hardcoded password.
- **Encrypted Channels:** Simulates pushing the compressed archives out to cloud providers (AWS S3, Mega) over standard HTTPS (Port 443) to blend in with normal web traffic.

IX. Defensive Engineering: Purple Team Agent

The Crown Jewel Feature

While the Operator Fleet breaks things, the Purple Team Agent fixes them. This agent represents the single largest leap over traditional adversary emulation frameworks.

The Purple Team Agent acts as a Senior Detection Engineer. It reads the execution logs of the Red Team and immediately builds defensive countermeasures.

- **Sigma Rule Generation:** Automatically generates production-ready Sigma rules. These rules target the specific Event IDs (e.g., Sysmon 13 for Registry, Event 4688 for Process Creation) triggered by the Red Team.
- **YARA Indicator Creation:** Generates YARA rules to catch the specific obfuscation patterns, base64 stubs, or file entropies of the payloads used by the Access Agent.
- **Deterministic Hashing:** Sigma rule UUIDs are generated via MD5 hashes of the technique IDs. This ensures that re-running the same threat actor produces the exact same rule IDs, allowing for seamless integration into production SIEMs without generating duplicates.

X. D3FEND Gap Analysis Engine

Identifying a gap in defenses is only half the battle. The Hive goes further by recommending exact architectural fixes.

- **MITRE D3FEND Mapping:** When a technique is identified as highly evasive or 'undetected', the system queries the MITRE D3FEND framework. It maps the offensive ATT&CK technique to its defensive counterpart.
- **Actionable Countermeasures:** Instead of just saying 'Pass the Hash failed', the system outputs: 'Deploy Credential Guard (D3-RRDD) and monitor Event ID 4624 Logon Type 9'. This gives SOC teams immediate remediation steps.

XI. Visual Interfaces: The Forensic CLI Sequence

The terminal interface (run.py) was custom-built to provide a cinematic, highly engaging user experience. It abandons boring standard-out logging for dynamic visual effects.

- **Hexadecimal Memory Scanning:** Upon launch, the terminal displays a rapid, scrolling matrix of randomized hexadecimal memory addresses, simulating a low-level memory scan of the target environment.
- **ANSI Glitch Effects:** The Hive's logo and phase transitions utilize ANSI color codes (reds, cyan, stark white) to create a 'glitching' visual effect, heavily inspired by modern cyberpunk/forensic aesthetics.
- **Real-Time Telemetry Ticker:** As agents execute, the CLI outputs a clean, timestamped ticker showing exact tool executions and memory states in real time.

XII. Visual Interfaces: The Tactical Dashboard

For a broader overview, The Hive features a standalone, zero-dependency web dashboard (dashboard.html). This dashboard is designed for SOC wall-monitors or Black Hat presentations.

- **Steel & Dark Grey Aesthetics:** The dashboard uses a highly refined, muted color palette (slate gray, tactical green, crimson red) to look premium and professional, avoiding cheap neon colors.
- **Canvas Matrix Rain:** The background features a custom HTML5 Canvas rendering a muted, slow-falling data rain. It provides motion without distracting from the data.
- **Scroll-Triggered Animations:** Using the IntersectionObserver API, the dashboard reveals agents, statistics, and pipeline phases smoothly as the user scrolls, creating a dynamic presentation experience.
- **Live State Emulation:** The dashboard features a live threat ticker and blinking operational states, simulating a live WebSocket connection to a running campaign.

XIII. Output Artifacts & File Structure

The Hive generates a highly structured output directory for every single simulation run.

- **report.md:** A human-readable markdown file summarizing the entire campaign, listing all techniques, extracted Sigma rules, and the final D3FEND gap analysis.
- **raw_results.json:** A massive, machine-readable JSON dump containing the entire LLM prompt history, execution logs, and API responses. Perfect for debugging or ingesting into external data lakes.
- **sigma_rules/:** A dedicated directory containing individual .yml files for every single Sigma rule generated during the run.
- **yara_rules/:** A dedicated directory containing the .yar files for static payload detection.

XIV. Deployment & Automation

The system is designed for zero-friction deployment on Windows.

- **start_dashboard.bat:** A one-click batch script that automatically launches a local Python HTTP server on port 8080 and opens the user's default browser to the dashboard.
- **stop_dashboard.bat:** A clean-up script that uses taskkill to gracefully shut down the background server by targeting its specific window title, preventing port-in-use errors.
- **Python Virtual Environment:** The entire tool runs within an isolated venv, ensuring that dependencies (fpdf2, markdown) do not conflict with the host system's global Python installation.

XV. Conclusion & Operational Impact

The Hive represents a paradigm shift in adversary emulation. By combining the natural language processing power of local LLMs with the structured execution of Atomic Red Team and the standardized defense formats of Sigma/YARA, it removes the human bottleneck from the threat intelligence lifecycle.

What previously required a dedicated Red Team, a Threat Intelligence Analyst, and a Detection Engineer working for weeks can now be accomplished by The Hive in under 5 minutes. It is the ultimate tool for continuous security validation.